The OpenClaw Bible

A field manual for running a real self-hosted AI agent

Dr Rushindra Sinha (Rushi)

September 2026 · written against OpenClaw 2026.9.x

How to read this

This is the manual I wish I had when I started. It comes from running my agent, Ares, every day since early 2026: across WhatsApp and Discord, a fleet of seven machines, dozens of scheduled jobs and a memory that now holds thousands of pages.

An honest warning first. I tore my hair out configuring this, and some days it still does more harm than good. It took 6 to 8 months of compounding before it became what it is now. And it isn't OpenClaw alone: the real power is the stack, Claude (and Claude Code) plus GPT and Codex, with OpenClaw as the surface that lets me turn it into anything. If you want something that works with less setup, look at Hermes Agent first. If you want to build your own system and own it, read on.

Every chapter has two parts. **The setup**, which is how OpenClaw works today. **The lesson**, which is what running it in real life taught me. The setup changes with every release, so check docs.openclaw.ai when in doubt. The lessons don't change much.

> Start locked down. Loosen slowly. An agent earns trust the same way a new hire does: one task done right at a time.

1. What OpenClaw is

OpenClaw is a self-hosted AI agent gateway. You run it on your own machine (a Mac mini, a laptop, a Linux box or a cloud server) and it connects the chat apps you already use to the AI models you choose.

The result is an assistant that lives in WhatsApp, Telegram, Discord, Slack, iMessage, Signal and many more, has access to your machine, and can do real work: files, browser, email, calendar, code, scheduled jobs.

- **Self-hosted.** Your data, config and memory stay on your machine.
- **Multi-channel.** One agent, many chat apps at the same time.
- **Multi-model.** Claude, GPT, Gemini, local models and more, with fallbacks.
- **Real access.** Shell, files, browser, devices, network, all governed by permission modes you control.

The lesson: think of it as a new employee with a laptop and your phone number, not a chatbot. That framing gets every other decision right, especially the safety ones.

2. What you need

REQUIREMENT	DETAILS
Node.js	24.16+ or 26.1+ (26 recommended). The installer sets it up if it’s missing.
Operating system	macOS, Linux or Windows (the Windows Hub app is the easiest Windows path).
AI access	An existing **Claude Code or Codex login**, or a provider API key. Onboarding can reuse a login you already have.
An always-on machine	Optional for testing, essential for a real assistant. A spare Mac mini is ideal.
Terminal comfort	You’ll read logs and edit a config file now and then.

The lesson: reusing a Claude subscription login is the simplest and most predictable way to pay for a personal agent. Per-token API billing is fine for experiments, but a busy agent’s bill is hard to predict.

3. Installing

The fastest path, in one command:

```
npx openclaw@latest
```

Or the installer script, which also installs Node if needed and launches onboarding:

```
curl -fsSL https://openclaw.ai/install.sh | bash      # macOS / Linux / WSL2
```

Onboarding offers **Quick start** (reuses the AI access it finds, verifies it with a real reply, saves your config and opens the dashboard) or **Custom setup** for every option. Then keep the gateway running in the background:

```
openclaw gateway install     # LaunchAgent on macOS, systemd on Linux
openclaw gateway status      # is it healthy?
openclaw dashboard           # web control panel
openclaw doctor              # diagnose and repair common problems
```

Come back to settings any time with `openclaw configure`.

The lesson: “the gateway is running” is not the same as “my agent works”. After any install, update or config change, check the thing you actually care about: send it a message from your phone and get a reply. `openclaw channels status` tells you more than `gateway status` does.

4. Where everything lives

```
~/.openclaw/                        runtime, managed by OpenClaw
├── openclaw.json                   main config
├── agents/<id>/agent/              per-agent auth, sessions, memory index
(SQLite)
├── media/                          files sent and received
└── workspace/                      default agent workspace (see below)

~/.openclaw/workspace/              the agent's home. Keep it private. Treat
it as memory.
├── AGENTS.md                       operating rules, plus a "## Tools" section
for local notes
├── SOUL.md                         persona, tone, boundaries
├── USER.md                         who you are, written as dated directives
├── IDENTITY.md                     name, vibe, emoji
├── MEMORY.md                       curated long-term memory
├── memory/YYYY-MM-DD.md            daily notes
├── BOOTSTRAP.md                    first-run ritual, delete when done
└── skills/                         your own skills (highest priority)
```

You can move the workspace anywhere with `agents.defaults.workspace`. Put it under git: it becomes a full history of how your agent has changed.

The lesson: never commit `~/.openclaw/` itself. That's where config, credentials and session history live.

5. The Soul Stack

> The secret is not in the skills. It's in the soul. Skills define **how** to use a tool. The soul defines **who** the agent is, **when** it acts, and **why**.

These files load into every conversation, so they're the most important words you'll write.

- **SOUL.md** sets personality, tone and hard boundaries.
- **AGENTS.md** is the employee handbook: rules, priorities, workflows. Local environment notes go in its `## Tools` section.
- **USER.md** describes you: preferences, communication style, time zone, written as short directives with dates, so an old preference can be marked as replaced instead of contradicted.
- **IDENTITY.md** holds the name, emoji and vibe. It matters more once you run several agents.
- **MEMORY.md** holds durable facts and decisions. See chapter 12.

A minimal SOUL.md:

```
# SOUL.md
You are Ares, chief of staff to Rushi.
Direct, concise, no filler. Answer first, detail second.

## Boundaries
- Draft anything. Never send, post, pay or delete without explicit approval.
- Never message anyone new on Rushi's behalf without permission.
- If you are not sure, say how sure you are and why.
```

The lessons:

- **Write the why, not just the rule.** "Never send without approval, because a wrong email to a partner can't be unsent" gets applied correctly in situations you never anticipated. A bare rule gets applied literally.
- **Keep these files lean.** They load every turn, cost tokens every turn, and very long files get truncated. Move reference material into separate docs the agent reads when needed, and keep the bootstrap files as an index.
- **When the agent makes a mistake, fix the file, not the chat.** A correction in chat is forgotten after the next reset. A line in AGENTS.md lasts.

6. Channels

OpenClaw supports a long list of chat apps. The common ones:

- **WhatsApp.** Pair it as a linked device (WhatsApp → Linked Devices → Link a Device) from onboarding or the dashboard. Give the agent its own number if you can.
- **Telegram.** Create a bot with @BotFather and add the token.
- **Discord.** Create a bot in the Developer Portal, enable the Message Content intent, invite it to your server.
- **Slack, iMessage, Signal, Matrix, Teams** and many more are documented under Channels in the docs.

In groups, the agent answers only when mentioned unless you change that.

The lessons:

- **Match the channel to the job.** WhatsApp can't stream, so the reader sees nothing until the whole reply is written. Keep WhatsApp replies short. Discord streams and renders tables and code, so long work goes there.
- **Big files go as links.** OpenClaw's WhatsApp limit is configurable (`channels.whatsapp.mediaMaxMb`), but in practice we see WhatsApp media fail above roughly 13 MB. We aim for 5 MB and send anything bigger as a Drive link.
- **Once you have two channels, every send must name its channel.** With one channel the default works. With two, an unaddressed send fails or goes to the wrong place.
- **Anything meant to be copied gets its own message.** An address, a command, an email draft: send it alone, with no labels inside, so it copies in one tap.

7. Secrets and keys

Never paste an API key into a chat. Use the **secrets** tool instead: the agent asks for a credential by name, you type it into a masked prompt, and it goes straight into the secret store. The value never appears in the chat, the transcript or the model's context.

```
openclaw secrets --help      # manage stored secrets from the CLI
```

Config fields that need a key can point at a stored secret instead of holding the raw value. The bundled `1password` skill works well if you already use 1Password.

The lessons:

- Never keep secrets inside a synced folder or notes vault. We once found 26 keys sitting in a synced notes folder. Rotate anything that has ever been exposed.
- Keep an index of *which* credentials exist and where (names only, never values) so the agent can check what it has before telling you it can't do something.

8. Models, thinking and cost

Switch models and effort from any chat:

```
/model              show or change the model
/think              set thinking depth (low → max)
/fast               fast mode on / off / auto
/status             model, usage, session info
/new  /compact      fresh session / summarise a long one
```

What we use in September 2026:

JOB	MODEL
Daily driver, most work	Claude Sonnet 5
Hard reasoning, important decisions, long builds	Claude Opus 5.5
Quick, cheap lookups	Claude Haiku 4.5
High-volume private work (summaries, extraction, transcription)	Local models via Ollama and Whisper

Configure **fallbacks** so a provider outage doesn't take your agent down.

The lessons:

- **Low effort costs more than it saves.** On low thinking and fast mode our agent confidently sent me across Shanghai to a ramen shop based on a 9-month-old listing. For anything with real-world consequences, set thinking to high and fast mode off. You can set this per agent with `thinkingDefault` and `fastModeDefault`.
- **Context length is the biggest cost driver.** Long sessions full of tool output burn tokens. Save what matters to memory and start fresh sessions regularly.
- **Push bulk, mechanical work to local models.** Our meeting-recording pipeline makes tens of thousands of local model calls a month at zero API cost.

9. Security and safety

> This is the most important chapter. Read it twice.

The agent runs with **your** user permissions. It can run commands, read and write your files, drive your browser, and message people as you. A misconfigured agent can delete files, leak credentials or send things you'd never send.

Permission modes control how shell commands get approved (`tools.exec.mode`):

MODE	WHAT HAPPENS
`deny`	No shell commands at all
`allowlist`	Only pre-approved commands; everything else is silently refused
`ask`	Pre-approved commands run; anything new asks you first
`auto`	Pre-approved commands run; new ones are reviewed and escalated to you when needed
`full`	No prompts. Only for a machine you fully trust the agent with.

```
openclaw config set tools.exec.mode ask
openclaw exec-policy show        # what's actually in effect
openclaw security audit          # check your setup for risky settings
```

Kill switches (memorise these):

```
openclaw gateway stop            # stops everything, immediately
openclaw gateway status          # confirm it's down
```

Then revoke or remove the AI credentials if you need to be sure it can't start again.

Rushi's method: start locked down, loosen slowly.

1. **Week 1:** chat only. No shell, workspace files only.
2. **Week 2:** `ask` mode. Approve every new command and watch what it wants to do.
3. **Week 3:** connect one channel, your own direct chat only.

4. **Week 4:** read-only calendar and email.
5. **Week 5 onward:** scheduled jobs, more channels, `auto` mode on a trusted machine.

The lessons:

- **Draft first, approve second, for everything outward-facing.** Emails, posts, payments, messages to new people. Approval is per item: if the content changes, the approval is void.
- **Approvals go stale.** Before acting on a “go” that came hours after the draft, check the thread again. You may have already sent it yourself. (We sent a partner the same email twice this way.)
- **Personal machines get “assist only”.** The agent can look and advise, never change things unattended.

10. Skills

Skills are Markdown instruction files (`SKILL.md`) that teach the agent how and when to use a tool. OpenClaw ships with hundreds of bundled skills; community skills live on ClawHub.

```
openclaw skills list                        # what's installed and ready
openclaw skills install @owner/<slug>       # install from ClawHub
openclaw skills install git:owner/repo      # install from a git repo
```

Your own skills go in `<workspace>/skills/` and take priority over everything else. The **Skill Workshop** lets the agent draft new skills as proposals that you review before they go live.

The lessons:

- A well-written SOUL.md with no skills beats fifty skills with a bad soul.
- **A skill the agent can't find is a skill you don't have.** With hundreds installed, vague descriptions get lost. Put the trigger phrases in the first line of the description. My skill-audit-router scores skills for exactly this.
- Before building something new, check whether a skill or script for it already exists. Offering to build what you already have is a real and embarrassing failure mode.

11. Automations and heartbeats

This is where an assistant becomes a chief of staff: work that happens without being asked.

- **Automations** are scheduled jobs: “brief me at 9 AM”, “check the inbox every 30 minutes”, “remind me in 20 minutes”. `openclaw automations list` shows them.
- **Heartbeat** is a built-in monitor that surfaces things quietly in your main session.
- **Hooks** react to events (a session reset, a tool call).
- **Standing orders** are instructions injected into every session.
- **Tasks** is the ledger of background work: `openclaw tasks list`.

Start with three automations: a morning brief, an evening wrap-up and an inbox sweep.

The lessons:

- **Every alert needs a gate.** A health check that fires on every run will be ignored within a week. Alerts should repeat only when something gets worse or after a real cooldown.
- **“Finished” isn’t proof.** A job is done when its output exists: the file, the sent message, the live page. Declare what each job should produce and check for it.
- **Mark test alerts as tests.** An alert that reads like an emergency should only ever mean one.

12. Memory

OpenClaw remembers by writing plain Markdown in the workspace. There is no hidden state: if it isn't on disk, the agent doesn't know it.

LAYER	FILE	PURPOSE
You	`USER.md`	Stable preferences and profile, as dated directives
Long-term	`MEMORY.md`	Durable facts and decisions, loaded every session
Daily	`memory/YYYY-MM-DD.md`	Running notes; today's and yesterday's load automatically
Dreams	`DREAMS.md`	Optional background consolidation (`/dreaming`)

The agent searches older memory on demand with its memory tools. To save something, just say "remember that…".

The lessons:

- **Keep MEMORY.md an index, not an archive.** Ours has a hard 8 KB ceiling: one line per item pointing to a topic file. The test for adding a line: "would losing this cause a wrong action next turn?"
- **Summaries lose things.** When a long conversation gets compacted, side threads can vanish without a trace. If something recent seems missing, search the full history before concluding it never happened.
- **Once memory outgrows files, add a knowledge graph.** We added GBrain (open source, Postgres-based) for searchable, linked pages about people, projects and decisions. It holds thousands of pages now. Watch its database connections: every agent session opens its own pool.

13. Multiple agents

One gateway can run several isolated agents, each with its own workspace, memory and login. **Bindings** route a channel account (a WhatsApp number, a Discord server) to a specific agent.

Agents coordinate by sending each other messages and by spawning **subagents** for focused jobs, which report back when done.

The lessons:

- Give each agent a clear lane and write the handoffs into each SOUL.md (“content work goes to Sam”).
- **Background work started from a chat turn can die when the turn ends.** For anything long, run it inside the turn or hand it to a scheduled automation that declares what it will produce.
- Before starting a deliverable, check whether another lane is already working on it. Two agents silently building the same thing wastes an evening.

14. Running a fleet

Once one machine works, you'll want more: a separate machine per business, a compute box, a cloud server.

- Put every machine on a private network (we use **Tailscale**) and reach them over SSH from one admin machine. No ports open to the internet.
- Each business gets its own OpenClaw install and its own WhatsApp number, so data never crosses.
- Put **local models** where they save the most: we run Qwen 3.5 for summarising recordings, Whisper (the Apple-silicon MLX build) for all voice transcription, and small embedding models for memory search.
- Log every infrastructure change with a way to roll it back.

The lessons:

- **Reachable isn't the same as working.** A machine that answers a ping can still have a dead chat channel. Check the real thing end to end.
- **Audit what's installed.** Months in, we found about 150 GB of local models that nothing had used in weeks.

15. Media and large files

- **Transcription:** Whisper runs locally for free. On Apple silicon, `mlx-whisper` with `large-v3-turbo` is fast and accurate.
- **Video and audio:** ffmpeg does almost everything. Check `ffmpeg -buildconf` before relying on filters such as text overlays; not every build includes them.
- **Large files:** keep them in cloud storage and share links. Google Drive supports HTTP range requests, so ffmpeg can seek into a multi-GB file without downloading it.

The lessons:

- Whisper sometimes writes Hindi audio in Urdu script. Use its timings and correct the text.
- YouTube has a daily upload cap separate from its API quota (`uploadLimitExceeded`). Wait 24 hours.
- MOV files often store their index at the end, so partial downloads can’t be read. Stream instead.

16. When things break

SYMPTOM	LIKELY CAUSE	FIX
Agent not replying	Gateway stopped or unhealthy	`openclaw gateway status`, then `openclaw doctor`
Gateway fine but chat dead	Channel disconnected	`openclaw channels status`; re-pair if needed
Broke it editing config	Invalid config	`openclaw config validate`, restore your backup
Agent “can’t” do something it did last week	Missing credential or tool	Check the credential index and skills list before accepting “can’t”
Memory database refuses connections	Too many sessions each holding a pool	Raise the database’s connection limit, close stale sessions
Same alert every 15 minutes	No alert gate	Add dedupe and cooldown
Confident but wrong answers	Low thinking or fast mode	Raise thinking, turn fast mode off
WhatsApp media won’t send	File too large	Compress under 5 MB or send a link

Always take a backup before big changes:

```
openclaw backup create
openclaw backup verify
```

17. Operating principles

1. **Accuracy over speed.** A fast wrong answer costs more than a slow right one.
2. **Say how sure you are.** Especially before sending someone somewhere or spending their money.
3. **Scope first, execute second.** “Here’s what I’ll do. Proceed?” for anything heavy.
4. **Give context, not just commands.** “Make a reel” gets a reel. “90-second comeback reel from map 3, silence trimmed, hero’s-journey arc” gets the reel you wanted.
5. **Proof, not claims.** Done means the output exists and you’ve seen it.
6. **Memory is everything.** If it wasn’t written down, it didn’t happen.
7. **Consider the human’s real constraints.** Don’t tell someone without a local SIM to call a restaurant.
8. **Fix the rule, not the instance.** Every mistake should become one line in a rules file so it never happens twice.

18. Quick reference

```
# Gateway
openclaw gateway status | restart | stop | install
openclaw status --deep          # overall health
openclaw channels status        # are the chat apps actually connected?
openclaw doctor --fix           # diagnose and repair
openclaw logs --follow          # live logs
openclaw dashboard              # web UI
openclaw update                 # update OpenClaw

# Safety
openclaw security audit
openclaw exec-policy show
openclaw backup create

# Work
openclaw automations list
openclaw tasks list
openclaw skills list
```

```
In chat:  /status  /model  /think  /fast  /new  /compact  /help
```

Links: docs.openclaw.ai · ClawHub · Rushi on X

> Start locked down. Loosen slowly. The agent earns trust the same way a new employee does: one successful task at a time.